

Animate Subject Tracks

Technical Guide

Animated 3D Movement Maps & Video — Pipeline & Rendering Reference

Workflow version 1.0.0 · Generated September 25, 2026

1. Overview

The **Animate Subject Tracks** workflow turns the GPS history of an EarthRanger subject group into an animated, terrain-aware 3D map. Each subject's track is cleaned into trajectory segments, stitched into one timestamped path per subject, draped onto a digital elevation model, and played back on a shared clock over satellite imagery.

The workflow produces an interactive HTML map (deck.gl, via pydeck) with a playback bar, an MP4 video recorded frame by frame from that map in a headless browser, GeoParquet copies of the relocations and trajectories, and a single-widget dashboard.

Note: The workflow has no groupers: `set_groupers` is configured with an empty list, so every subject in the group is animated together on one map.

2. Dependencies & Prerequisites

2a. EarthRanger

Observations are pulled live from EarthRanger through the connection chosen in **Connect to EarthRanger** (`set_er_connection`). The user supplies a subject group name; every subject in that group is included.

2b. External Tile Services

The 3D terrain and its texture are fetched at run time from public tile services:

Service	URL template	Used for
AWS Terrarium elevation	<code>https://s3.amazonaws.com/elevation-tiles-prod/terrarium/{z}/{x}/{y}.png</code>	Terrain mesh and sampling ground height for draping
Esri World Imagery	<code>https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}</code>	Satellite texture draped on the terrain mesh

Both are needed by the interactive map and by the video renderer, so the machine running the workflow (and anyone viewing the HTML map) needs internet access.

2c. Headless Browser

Video rendering uses Playwright's Chromium. The first time `render_animation` runs in an environment it runs `playwright install chromium`, which downloads the browser. Encoding uses the `ffmpeg` binary bundled with `imageio-ffmpeg`.

3. Data Ingestion Pipeline

Step (task id)	Task	What it does
<code>subject_observations</code>	<code>get_subjectgroup_observations</code>	Fetches observations for the subject group over the time range, with <code>filter: clean</code> (only fixes with exclusion flag 0), observation details and subject-source details. <code>raise_on_empty: false</code> returns an empty frame instead of failing.

Step (task id)	Task	What it does
subject_reloc	process_relocations	Converts observations to relocations, keeps the subject name, colour, sex, subtype and source columns, and drops fixes at the known junk coordinates (180, 90), (0, 0) and (1, 1).
convert_to_trajs	relocations_to_trajectory	Builds trajectory segments between consecutive fixes and applies the user's Trajectory Segment Filter .
rename_traj_cols	map_columns	Renames extra__hex → hex_color, extra__name → subject_name, and the sex, subtype and created-at columns. Missing columns are ignored.
persist_relocs_geoparquet	persist_df_wrapper	Writes relocations to relocations_<hash>.geoparquet, sanitising nested objects for Arrow.
persist_trajs_geoparquet	persist_df_wrapper	Writes trajectories to trajectories_<hash>.geoparquet.

3a. Trajectory Segment Filter Defaults

Limit	Minimum	Maximum
Segment length	0.001 m	100 000 m
Segment duration	1 s	172 800 s (2 days)
Segment speed	0.01 km/h	500 km/h

4. Terrain

4a. Elevation Decoder & Exaggeration

`create_elevation_decoder` builds the RGB-to-elevation decoder for Terrarium tiles (`r_scaler` 256, `g_scaler` 1.0, `b_scaler` 1/256, `offset` -32768). `deck.gl`'s `TerrainLayer` has no elevation-scale property, so **Exaggeration** is applied by multiplying every decoder term. The same decoder feeds both the terrain mesh and the track sampling, so the tracks always sit on the exaggerated surface.

4b. Terrain Layer

`create_terrain_layer` defines the 3D terrain mesh from the Terrarium tiles with the Esri World Imagery texture, zoom range 0–15 and wireframe off.

4c. Draping Tracks on Terrain

`create_terrain_sampling` configures elevation sampling with the user's **Offset** (default 30 m) and **Ground Elevation** fallback (default 1000 m). `drape_trips_on_terrain` then sets the z value of every track vertex to the sampled ground height plus the offset. All vertices are sampled in one batched call, so each elevation tile is fetched once. If sampling fails, every vertex uses the constant ground elevation.

Note: `custom_basemap_urls` (`set_basemap_urls`) and `basemap_option` (`set_basemap_option`) are evaluated but not consumed by any later step: the terrain layer and sampling take their tile URLs directly.

5. Animation Data

Step (task id)	Task	What it does
ensure_wgs	convert_crs	Reprojects trajectories to EPSG:4326.
trajs_trips	trajectory_to_trips	Stitches each subject's segments (grouped by <code>groupby_col</code>) into one lon/lat LineString with per-vertex timestamps, keeping <code>subject_name</code> and <code>hex_color</code> .
drape_terrain	drape_trips_on_terrain	Adds terrain heights to every vertex (section 4c).
rgba_hex	add_rgba_from_hex	Converts each subject's EarthRanger hex colour to an RGBA <code>rgba_color</code> column for the trail and legend.
zoom_to_envelope	envelope_gdf	Takes the bounding envelope of all trips (expansion factor 1.0).
trips_view_state	compute_view_state_from_gdf	Computes the centre and zoom that fit the envelope (max zoom 15), plus the user's Pitch and Bearing .

6. Animated Map

6a. Trips Animation

`create_trips_animation` defines how each track plays back. Each subject shows a bright comet tail in its own colour covering 95% of the timeline (`comet_ratio 0.95`), over a white history line of the whole path travelled so far (opacity 0.85, fading). The user picks the **Marker icon** drawn at each subject's current position:

Marker	Rendering
Dot (default)	ScatterplotLayer circle, 6 px radius, track colour, 1.5 px white outline
3D animal	Preset ScenagraphLayer model: elephant, giraffe, cheetah, leopard or lion; size and lighting adjustable
3D model (custom)	ScenagraphLayer from a user GLB (URL, data URI or path). Faces the direction of travel, with heading smoothing, optional terrain-slope pitch and size clamps (12–75 px by default)
None	Trails only

6b. Trips Layer

`create_trips_layer` draws the draped trips as a deck.gl TripsLayer: `timestamps` as the time accessor, `rgba_color` as the colour, 2.15 px width (clamped to 1–4 px), rounded caps and joints. The legend is titled **Subjects**, labelled by `subject_name` and sorted ascending. `animate_layer` attaches the trips animation to this layer.

6c. Playback Controls & Timeline

`create_playback_controls` configures the playback bar (play, restart, scrubber, clock, data time, speed button cycling 0.5×/1×/2×/4×, bottom or top). `create_timeline_animation` builds the shared clock: playback length **Duration S** (default 30 s), a 30 fps cap for the interactive map, and optional **Auto Rotate Speed**.

6d. Drawing & Saving

`draw_animated_map` combines the animated trips layer, the terrain tile layer, the view state and the timeline into a static HTML page (max zoom 15, legend bottom-right). Every animated layer is rebased onto a common timeline

starting at its earliest timestamp, so all subjects stay in step. Each layer gets a stable deck.gl id that the page's animation script uses to find it. persist_text writes the page to animated_map.html.

7. Video Rendering

render_animation records animated_map.html to animation.mp4:

- Launches headless Chromium with ANGLE WebGL (GPU when available; SwiftShader software rendering when gl: software).
- Reads the timeline span and the animation's natural length. With **Duration** on Auto the video matches **Duration S**; otherwise it uses the fixed seconds. Frames = fps × seconds (30 fps; 900 frames by default).
- Computes the whole per-frame camera path in the page from the chosen **Camera**, fitted to the video size and the map's pitch and bearing.
- Splits frames into contiguous chunks across parallel browser pages. workers: auto uses half the CPU cores, capped by GPU sharing, free memory and frame count.
- For each frame: sets the view state, renders the animation at that time, waits two animation frames plus any pending tile requests (up to 8 s, then 30 ms settle), and screenshots the map canvas.
- Encodes the numbered frames with ffmpeg: libx264, CRF 18, veryfast preset, yuv420p, dimensions rounded to even numbers, +faststart for web playback.

7a. Camera Options

Camera	Behaviour	Main settings
Static	One view fitting all animated data	Zoom Offset
Follow the action	Frames recent movement and re-fits smoothly	Follow Window 0.1, Follow Smoothing 0.25, Heading Lock, Fit Padding 80 px
Orbit	Circles the centre of the data	Rounds 1.0
Fit everything so far	Zooms to keep all data shown so far in frame	Fit Padding 80 px
Cinematic fly-through	Intro from the whole scene, then a turning, tilted follow	Intro Frac 0.12, Lead Frac 0, Bearing Mode rotate/heading/fixed, Rotate Deg 45
Keyframes	Flies through waypoints; inline, from a subject, or from a file	Source (subject / file), Keyframe Easing smooth/linear/spline
Fly to & around	Flies to a point, then circles it	Lon/Lat, Altitude, Range, Tilt, Heading (blank = from data or map), Fly Frac 0.25, Rounds 1.0

Keyframe files may be a .json list of {lon, lat, t?, zoom?, pitch?, bearing?} objects, a .geojson of Point features (extras read from properties), or a .csv/.tsv with lon/lat columns. At least two keyframes are needed. Missing zoom, pitch or bearing values are interpolated between neighbouring keyframes, or fall back to the whole-scene framing and the map's pitch and bearing.

7b. Fixed Encoding Settings

Setting	Value
Frame rate	30 fps
Clip range	Whole timeline (start_frac 0, end_frac 1)

Setting	Value
Frame capture	JPEG, quality 92, device scale factor 1
Encoder	libx264, CRF 18, preset veryfast
Capture timing	settle 30 ms, tile timeout 8 000 ms, 3D-model load timeout 30 000 ms

8. Interactive Dashboard

`create_map_widget_single_view` wraps `animated_map.html` as the **Subject movements** widget. `gather_dashboard` assembles it with the workflow details, time range and (empty) groupers. The video is not a dashboard widget; it is written to the results folder.

9. Output Files

All outputs are written to `ECOSCOPE_WORKFLOWS_RESULTS`.

File	Written by	Contents
<code>relocations_<hash>.geoparquet</code>	<code>persist_relocs_geoparquet</code>	Cleaned GPS fixes
<code>trajectories_<hash>.geoparquet</code>	<code>persist_trajs_geoparquet</code>	Filtered trajectory segments with renamed columns
<code>animated_map.html</code>	<code>map_urls</code>	Interactive animated 3D map
<code>animation.mp4</code>	<code>create_animation</code>	H.264 video of the animation

10. Workflow Execution Logic

10a. Skip Conditions

Scope	Conditions	Effect
Default (all tasks)	<code>any_is_empty_df</code> , <code>any_dependency_skipped</code>	A task is skipped if any input dataframe is empty or any upstream task was skipped. An empty EarthRanger result therefore skips the whole chain rather than raising an error.
<code>create_animation</code>	<code>any_dependency_skipped</code>	Its input is a file path, not a dataframe, so only the upstream-skip check applies.
<code>animated_map_widget</code>	<code>never</code>	Always runs, so the dashboard always has its map widget.

10b. Data Flow Summary

Stage	Flow
Ingestion	Observations → relocations → trajectories → renamed columns → GeoParquet
Terrain	Elevation decoder → terrain layer + terrain sampling
Animation data	EPSG:4326 → trips per subject → draped on terrain → RGBA colours

Stage	Flow
Framing	Trips envelope → view state (+ pitch, bearing)
Map	Trips animation + trips layer → animated layer; controls → timeline; all → draw_animated_map → HTML
Video	HTML → headless Chromium frames → ffmpeg → MP4
Dashboard	HTML → map widget → gather_dashboard

11. Software Versions

Package	Version	Role
ecoscope-platform	>=2.15.0, <2.16.0	Core runtime and tasks (EarthRanger I/O, preprocessing, CRS, persistence, dashboard)
ecoscope-workflows-ext-custom	0.1.0rc14.*	Map layer definitions and persist_df_wrapper
ecoscope-workflows-ext-ste	0.0.0rc1.*	Envelope, view state and hex-to-RGBA tasks
ecoscope-workflows-ext-mep	1.0.4.*	Terrain, trips, animated map and video rendering tasks
pydeck	0.9.2	deck.gl map generation
opentelemetry-sdk	>=1.20.0, <2.0.0	Tracing
wt-compiler	>=0.8.2, <0.9	Compiles spec.yaml (development only)

Packages are distributed via the **prefix.dev** conda channels and conda-forge. The runtime environment is managed by **pixi**.